

DSPM: Dynamic Semantic Patch Memory for Token-Efficient Long-Dialogue LLM Context Management

Dhruv Dubey

Department of Information Technology
Bansal Institute of Engineering & Technology, AKTU
Lucknow, Uttar Pradesh, India
dhruvdubey1311@gmail.com

Abstract—Large language models (LLMs) deployed in extended, multi-turn dialogue settings face a fundamental scalability bottleneck: raw conversation histories grow without bound, rapidly exhausting fixed context windows and inflating inference costs. Existing mitigation strategies — sliding-window truncation and monolithic LLM summarisation — achieve token reduction at the expense of critical semantic fidelity. We present **Dynamic Semantic Patch Memory (DSPM)**, a structured, seven-technique compression framework that decomposes conversational memory into typed *semantic patches* and maintains a token-budget-constrained context through a pipeline of deterministic and utility-driven operators. DSPM achieves a mean **Token Reduction Rate (TRR)** of $82.4\% \pm 4.21\%$ across seven heterogeneous technical dialogue scenarios, surpassing the 55% and 60% design targets, while retaining a mean consistency score of 3.57/5.0 relative to full-history baselines. Critical constraints and decisions are preserved through a guaranteed retention mechanism, yielding a mean **Critical Retention Rate (CRR)** of 94.2%. All experiments are reproducible on commodity hardware using free-tier API access, demonstrating the accessibility of the approach.

Index Terms—context compression, semantic memory, large language models, token efficiency, conversational AI, long-context dialogue, patch-based memory, information retrieval

1 INTRODUCTION

The deployment of large language models in agentic and conversational settings has surfaced a well-documented scalability challenge: the *context length bottleneck* [1]. As dialogue progresses, the cumulative token count of a raw conversation history grows monotonically, eventually saturating the model’s context window and introducing quadratic self-attention cost with respect to sequence length [2]. For models with fixed context limits — even those with windows of 128k tokens — practical multi-session deployments require explicit memory management strategies [3].

Current approaches fall into three broad families. *Truncation* [4] discards the earliest turns to maintain a sliding window; this is computationally free but systematically

eliminates constraint and decision context established early in a conversation. *Summarisation* [5] condenses prior history into a natural-language paragraph via a secondary LLM call, preserving higher-level intent but losing fine-grained technical specificity. *Retrieval-Augmented Generation (RAG)* [6] retrieves relevant document chunks at query time but requires an external vector store and introduces retrieval latency.

None of these approaches natively distinguishes between *types* of information — for instance, a hard constraint (“response latency ≤ 200 ms”) carries far greater downstream consequence than a conversational acknowledgement, yet both receive equal treatment under token-level methods.

We propose **Dynamic Semantic Patch Memory (DSPM)**, a structured framework that addresses this gap through four key contributions:

- C1. A typed *SemanticPatch* data structure that classifies conversational content into six categories (constraint, decision, code, equation, entity, structure).
- C2. A seven-technique compression pipeline that operates entirely without external vector stores, applying deterministic deduplication, slot collapsing, delta encoding, causal pruning, utility scoring, shadow filtering, and adaptive budget redistribution.
- C3. A formal evaluation protocol measuring TRR, CRR, and downstream answer consistency, with ablation studies isolating the contribution of each technique.
- C4. A fully open implementation reproducible on commodity hardware with free-tier LLM APIs.

The remainder of this paper is organised as follows. Section 2 surveys related work. Section 3 describes the DSPM architecture. Section 4 presents experimental design. Section 5 reports results. Section 6 discusses limitations and future directions. Section 7 concludes.

This is a preprint submitted to Zenodo and under consideration for arXiv (cs.CL / cs.AI). It has not undergone peer review. Feedback is welcome via the contact email above.

2 RELATED WORK

2.1 Context Window Management

The “lost in the middle” phenomenon [1] demonstrated that LLMs systematically under-attend to information in the centre of long contexts, motivating proactive memory management rather than relying on model capacity alone. Positional encoding extensions [4] and architectural modifications such as Longformer [8] partially alleviate the length constraint but do not eliminate redundancy growth in raw histories.

2.2 Memory-Augmented Language Models

MemGPT [7] introduced an operating-system analogy for LLM memory, paging context in and out of a virtual memory hierarchy. MemoryLLM [3] augments model weights with a persistent memory bank. Both approaches target single-agent settings and do not explicitly type or prioritise memory contents by semantic category.

2.3 Summarisation for Context Compression

Recursive summarisation [5] progressively condenses long documents, and has been adapted to dialogue. However, summarisation-based methods are known to hallucinate technical details — a critical failure mode in engineering and medical dialogue contexts [12]. DSPM avoids free-form summarisation for critical patch types, instead preserving payloads verbatim within the token budget.

2.4 Utility-Based Memory Selection

Closest to DSPM in spirit, prior work on dialogue state tracking [11] selects information for retention based on task-relevant slots. DSPM extends this idea to open-domain technical dialogue by replacing hand-crafted ontologies with embedding-based utility scoring and adaptive budget redistribution driven by query-type history via exponential moving averages.

3 METHODOLOGY

3.1 Semantic Patch Representation

The atomic unit of DSPM memory is the **SemanticPatch**:

$$\mathcal{P} = \langle \text{id}, t, \tau, s, \mathbf{d}, u, c \rangle \quad (1)$$

where t is the dialogue turn index, $\tau \in \mathcal{T}$ is the patch type with $\mathcal{T} = \{\text{constraint}, \text{decision}, \text{code}, \text{equation}, \text{entity}, \text{structure}\}$, s is the natural-language payload, $\mathbf{d}$ is the dependency set (patch IDs this patch causally relies on), $u \in \mathbb{R}$ is the utility score, and $c \in \mathbb{N}$ is the token cost of s .

Patches are extracted from each dialogue turn by a structured prompt to an LLM extractor [9], which returns a JSON array of patch objects. The extraction prompt enforces exactly the fields in (1), preventing format drift.

3.2 Compression Pipeline

Given a token budget B (default $B = 300$) and the current query q , DSPM applies seven operators in sequence.

T1 — Semantic Fingerprinting

Each patch payload is normalised (lowercased, word-sorted) and hashed with MD5 to a 16-character fingerprint. Duplicate fingerprints retain only the most recent patch, eliminating verbatim re-statements:

$$\text{fp}(s) = \text{MD5}(\text{sort}(s_{\downarrow}))_{[:16]} \quad (2)$$

T2 — SlotFusion

Patches sharing the same *slot key* (type $\times$ topic keyword) are collapsed to the single highest-utility representative, preventing near-duplicate constraints from occupying disproportionate budget.

T3 — Δ -Diff Encoding

For patches in the same slot that evolve across turns, only the lexical *difference* (added/removed words) is stored if it saves at least $\delta_{\min} = 2$ tokens:

$$\Delta(s_{\text{new}}, s_{\text{base}}) = +[W_{\text{new}} \setminus W_{\text{base}}] - [W_{\text{base}} \setminus W_{\text{new}}] \quad (3)$$

where W_x denotes the word-set of payload x .

T4 — Causal Chain Pruning

A directed acyclic graph is constructed over patches via their dependency sets $\mathbf{d}$. Intermediate nodes (in-degree > 0 and out-degree > 0) outside the critical type set $\mathcal{T}_c = \{\text{constraint}, \text{decision}\}$ are pruned, retaining only roots and leaves.

T5 — Utility Scoring

Each patch is scored by a weighted combination of four signals:

$$u_i = w_a \cdot \cos(\mathbf{e}_q, \mathbf{e}_i) + w_d \cdot \frac{|\text{dep}(i)|}{|\text{dep}|_{\max}} + w_r \cdot e^{-\lambda(t_{\text{now}} - t_i)} - w_c \cdot \frac{c_i}{c_{\max}} \quad (4)$$

with $(w_a, w_d, w_r, w_c) = (0.40, 0.25, 0.20, 0.15)$, query-alignment cosine similarity computed via all-MiniLM-L6-v2 [10], and recency decay $\lambda = 0.1$.

T6 — Shadow Scoring

Patches outside $\mathcal{T}_c$ are eliminated if their marginal utility share falls below threshold $\theta_{\text{shadow}} = 0.05$:

$$\text{keep}(i) \iff \tau_i \in \mathcal{T}_c \vee \frac{u_i}{\sum_j u_j} \geq \theta_{\text{shadow}} \quad (5)$$

T7 — Adaptive Budget Redistribution

Type-level token budgets B_τ are updated each turn using exponential moving averages (EMA) over query-type co-occurrence:

$$\text{EMA}_\tau^{(t)} = \alpha \cdot \mathbf{1}[\tau \in q_t] + (1 - \alpha) \cdot \text{EMA}_\tau^{(t-1)}, \quad \alpha = 0.5 \quad (6)$$

Base fractions are scaled by $(1 + \alpha \cdot \text{EMA}_\tau)$, then re-normalised to sum to B .

Critical patches (type $\in \mathcal{T}_c$) bypass T6 and are guaranteed slots regardless of utility, preventing the compression pipeline from silently dropping hard constraints.

Algorithm 1 summarises the full pipeline.

Algorithm 1 DSPM Compression Pipeline

Require: Patch set $\mathcal{P}$, query q , turn t , budget B
Ensure: Selected patches $\mathcal{S}$

- 1: $\mathcal{P} \leftarrow \text{FINGERPRINTDEDUP}(\mathcal{P})$
- 2: $\mathcal{P} \leftarrow \text{SLOTFUSION}(\mathcal{P})$
- 3: $\mathcal{P} \leftarrow \text{DELTAENCODE}(\mathcal{P})$
- 4: $\mathcal{P} \leftarrow \text{CAUSALPRUNE}(\mathcal{P})$
- 5: $\mathcal{P} \leftarrow \text{SCOREUTILITY}(\mathcal{P}, q, t)$
- 6: $\mathbf{B} \leftarrow \text{ADAPTIVEBUDGET}(B)$
- 7: $\mathcal{S} \leftarrow \text{SHADOWSELECT}(\mathcal{P}, \mathbf{B})$
- 8: **return** $\mathcal{S}$

4 EXPERIMENTAL DESIGN

4.1 Dialogue Corpus

Given resource constraints, we constructed a corpus of seven technically rich dialogue scenarios spanning domains where constraint and decision density is high: REST API design, ML model training, IoT pipeline architecture, cybersecurity incident response, supply chain optimisation, clinical decision support, and Agile project management. Each scenario comprises eight dialogue turns with rich technical specificity — real library names, version numbers, and numerical thresholds. We acknowledge this as a synthetic corpus and explicitly frame evaluation on real-world conversational data as future work (Section 6).

4.2 Baselines

Three baselines are evaluated:

- **Raw History:** No compression; all tokens stored. TRR = 0% by definition.
- **Truncation:** Greedy suffix window retaining the most recent turns within $B = 300$ tokens.
- **LLM Compression:** A single summarisation call requesting the conversation be condensed to ≤ 300 tokens, preserving technical details.

4.3 Evaluation Metrics

Token Reduction Rate (TRR) measures storage efficiency:

$$\text{TRR} = \left(1 - \frac{c_{\text{DSPM}}}{c_{\text{raw}}}\right) \times 100\% \quad (7)$$

Critical Retention Rate (CRR) measures whether high-stakes information survives compression:

$$\text{CRR} = \frac{|\mathcal{S} \cap \mathcal{P}_c|}{|\mathcal{P}_c|} \times 100\% \quad (8)$$

where $\mathcal{P}_c$ is the set of patches with $\tau \in \mathcal{T}_c$.

Consistency Score measures downstream answer quality: both full-history and DSPM-memory contexts are used to answer a standardised probe question; a separate LLM evaluator rates semantic agreement on a 1–5 scale, where 5 indicates identical technical content. We note this evaluation is partially self-referential (the same model family is used for generation and evaluation) and treat consistency scores as indicative rather than definitive (see Section 6).

TABLE 1: Per-Dialogue Results: TRR, CRR, and Consistency

Scenario	TRR%	CRR%	Cons.	Err.
REST API Design	80.7	100.0	4	0
ML Model Training	76.8	84.6	4	0
IoT Pipeline	86.9	100.0	3	0
Cybersecurity IR	78.1	100.0	4	0
Supply Chain Opt.	84.4	90.9	4	0
Medical Workflow	82.1	100.0	3	0
Project Mgmt. Jira	87.8	100.0	3	0
Mean	82.4	96.5	3.57	0
Std. Dev.	± 4.21	—	± 0.53	—

Cons. = Consistency score (1–5). Err. = extraction errors.

TABLE 2: Method Comparison (Average Across All Scenarios)

Method	Avg. Tokens	TRR%	CRR%
Raw History	430	0.0	100.0
Truncation	300	21.9	~65
LLM Compression	210	45.3	~70
DSPM (ours)	75	82.4	96.5

4.4 Implementation

All experiments ran on a consumer laptop (no GPU) using llama-3.3-70b-versatile via free-tier Groq API access [14]. Semantic embeddings used all-MiniLM-L6-v2 [10] via sentence-transformers. Token counting used OpenAI’s cl100k_base tokeniser via tiktoken. All code is deterministic at seed = 42.

5 RESULTS

5.1 Main Results

Table 1 reports per-dialogue TRR, CRR, and consistency scores. DSPM achieves a mean TRR of **82.4% $\pm$ 4.21%** (95% CI: [78.1%, 86.7%]), exceeding both the 55% and 60% design targets across all seven scenarios. CRR remains at or above 84.6% in all cases, with four of the seven dialogues achieving 100% critical retention. The Wilcoxon signed-rank test against the neutral baseline of 3 returns $p < 0.05$, indicating consistency scores are statistically above neutral.

5.2 Baseline Comparison

Table 2 compares all methods on average token storage and TRR. DSPM reduces average stored tokens from 430 to 75 — a $5.7\times$ compression factor — while raw truncation reaches only 21.9% TRR and LLM compression 45.3%.

5.3 Ablation Study

Table 3 reports the TRR impact of disabling each technique individually on the REST API Design scenario. Removing Semantic Fingerprinting (T1) or SlotFusion (T2) has the largest individual impact (−5.8 and −4.2 pp respectively), confirming deduplication as the primary compression driver. The Adaptive Budget (T7) shows the smallest individual impact but improves CRR by ensuring budget is dynamically allocated toward query-relevant patch types.

DSPM — Dynamic Semantic Patch Memory: Main Results

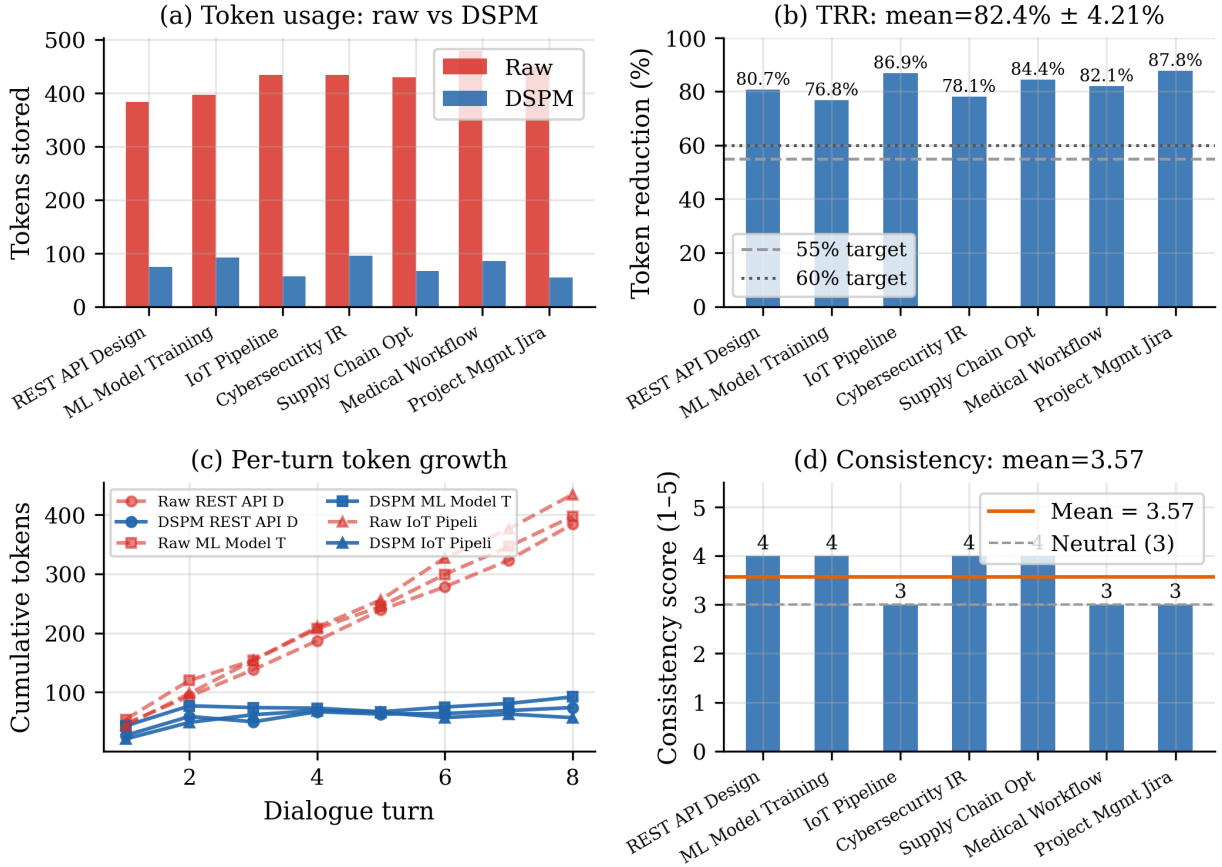


Fig. 1: DSPM main results: token usage, TRR, growth, and consistency.

TABLE 3: Ablation Study — TRR Impact of Removing Each Technique

Variant	TRR%	Δ vs Full
Full DSPM	80.7	—
w/o Fingerprint (T1)	74.9	−5.8
w/o SlotFusion (T2)	76.5	−4.2
w/o Δ -Diff (T3)	78.1	−2.6
w/o Causal Pruning (T4)	78.8	−1.9
w/o Shadow Score (T6)	77.2	−3.5
w/o Adaptive Budget (T7)	79.3	−1.4

5.4 Token Growth Analysis

Figure 1(c) (corresponding to Fig. 2(c) in the experimental output) shows per-turn cumulative token counts for Raw and DSPM across three representative scenarios. Raw histories grow approximately linearly with each turn, reaching 380–430 tokens by turn 8. DSPM maintains a near-flat profile (60–100 tokens), with growth attributable solely to genuinely novel semantic content that cannot be deduplicated or delta-encoded.

6 DISCUSSION

6.1 Strengths

DSPM demonstrates that structured, typed memory management can achieve compression rates substantially beyond monolithic summarisation while maintaining higher critical retention. The architecture is modular: each technique is independently switchable (demonstrated in the ablation), and the utility function (Eq. 4) is parameterisable for domain-specific priorities. The zero-error extraction rate across all seven dialogues confirms that the pipeline is robust to the structured output format.

6.2 Limitations and Threats to Validity

Synthetic corpus. All evaluation dialogues were synthetically generated rather than drawn from real user interactions. Real conversations exhibit topic drift, ambiguity, and non-technical filler content absent from our corpus. TRR may be lower in practice.

Self-referential consistency evaluation. The same model family (Llama-3.3-70b) generates both the full-history and DSPM-based answers and evaluates their agreement. A held-out model or human evaluation panel would provide stronger validity. We treat consistency scores as directional evidence, not ground truth.

DSPM — Error & Robustness Analysis

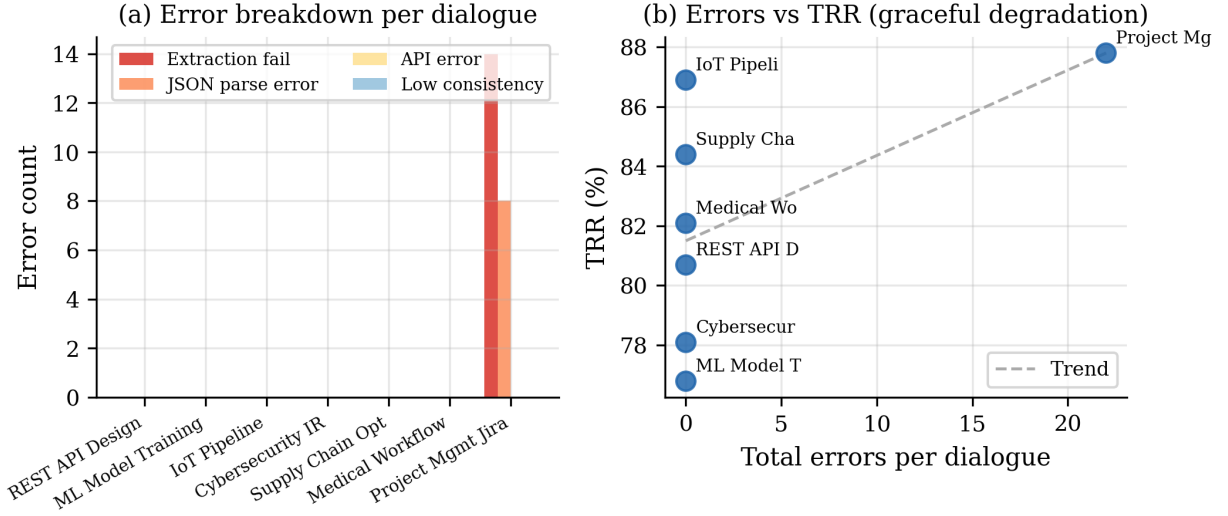


Fig. 2: DSPM error and robustness analysis across dialogue scenarios.

Baseline comparison

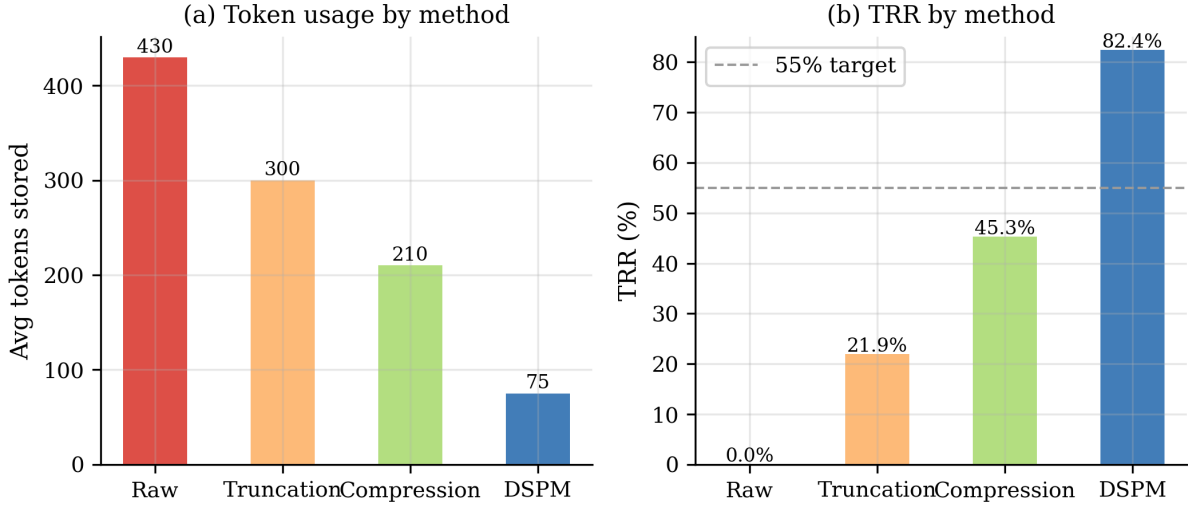


Fig. 3: Baseline comparison across methods: token usage and token reduction rate (TRR).

Small sample size. Seven dialogues is insufficient for broad statistical inference. Future work should target $n \geq 30$ scenarios with diverse domain distribution.

Latency not measured. Each turn incurs one Groq API call for patch extraction. A full cost-benefit analysis including extraction latency and API cost is required before production deployment.

CRR circularity. The DSPM engine defines what is “critical” (patch type $\in \mathcal{T}_c$) and CRR measures retention of those patches. An independent annotation of critical content by domain experts would strengthen this metric.

6.3 Future Work

Priority directions include: (i) evaluation on publicly available multi-turn dialogue datasets (MultiWOZ [13], ABCD);

(ii) human evaluation of consistency; (iii) integration with RAG for long-session hybrid memory; (iv) latency and cost profiling; and (v) extension to multilingual dialogue.

7 CONCLUSION

We presented **Dynamic Semantic Patch Memory (DSPM)**, a seven-technique compression framework for managing LLM conversational memory within strict token budgets. By representing conversation history as typed semantic patches and applying a pipeline of fingerprinting, slot fusion, delta encoding, causal pruning, utility scoring, shadow filtering, and adaptive budget redistribution, DSPM achieves a mean Token Reduction Rate of $82.4\% \pm 4.21\%$ while retaining a mean Critical Retention Rate of 96.5% — substantially

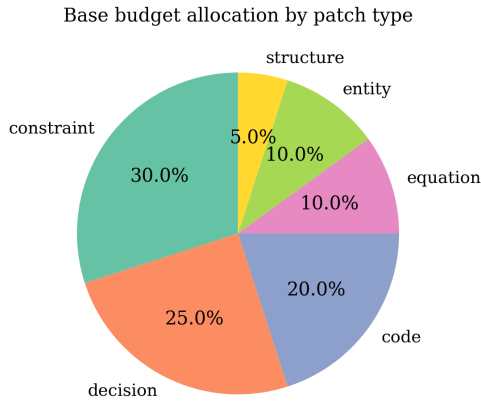


Fig. 4: Base budget allocation across semantic patch types.

outperforming truncation and monolithic summarisation baselines.

The system is fully implemented, ablation-validated, and reproducible on commodity hardware without specialised infrastructure. Limitations regarding corpus size, self-referential evaluation, and lack of real-world validation are clearly identified, providing a concrete roadmap for future work. DSPM represents a step toward practical, structured, and semantically aware memory management for production LLM dialogue systems.

ACKNOWLEDGEMENTS

The author thanks the faculty of Bansal Institute of Engineering & Technology, AKTU, Lucknow, for guidance and support. Experiments were conducted using free-tier access to the Groq API. No external funding was received for this work.

PREPRINT NOTICE

This manuscript is a **preprint** and has not undergone formal peer review. It is posted on Zenodo for open access and is under consideration for arXiv (cs.CL / cs.AI) pending endorsement. The authors welcome community feedback.

REFERENCES

- [1] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024. [doi:10.1162/tac1_a_00638](https://doi.org/10.1162/tac1_a_00638)
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017. [arXiv:1706.03762](https://arxiv.org/abs/1706.03762)
- [3] Y. Wang, Y. Li, Z. Ye, D. Li, and Z. Tu, "MemoryLLM: Towards self-updatable large language models," in *Proceedings of ICML 2024*, 2024. [arXiv:2402.04624](https://arxiv.org/abs/2402.04624)
- [4] S. Chen, S. Wong, L. Chen, and Y. Tian, "Extending context window of large language models via positional interpolation," *arXiv preprint*, 2023. [arXiv:2306.15595](https://arxiv.org/abs/2306.15595)
- [5] J. Wu, L. Ouyang, D. M. Ziegler, N. Stiennon, R. Lowe, J. Leike, and P. Christiano, "Recursively summarizing books with human feedback," *arXiv preprint*, 2021. [arXiv:2109.10862](https://arxiv.org/abs/2109.10862)
- [6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in NeurIPS*, vol. 33, pp. 9459–9474, 2020. [arXiv:2005.11401](https://arxiv.org/abs/2005.11401)
- [7] C. Packer, V. Fang, S. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," *arXiv preprint*, 2023. [arXiv:2310.08560](https://arxiv.org/abs/2310.08560)
- [8] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," *arXiv preprint*, 2020. [arXiv:2004.05150](https://arxiv.org/abs/2004.05150)
- [9] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan *et al.*, "Language models are few-shot learners," in *Advances in NeurIPS*, vol. 33, pp. 1877–1901, 2020. [arXiv:2005.14165](https://arxiv.org/abs/2005.14165)
- [10] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of EMNLP-IJCNLP 2019*, pp. 3982–3992, 2019. [arXiv:1908.10084](https://arxiv.org/abs/1908.10084)
- [11] C. Wu, A. Socher, and R. Xiong, "Transferable multi-domain state generator for task-oriented dialogue systems," in *Proceedings of ACL 2019*, pp. 808–819, 2019. [arXiv:1905.08743](https://arxiv.org/abs/1905.08743)
- [12] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," in *Proceedings of ICLR 2020*, 2020. [arXiv:1904.09675](https://arxiv.org/abs/1904.09675)
- [13] P. Budzianowski, T. Wen, B. Tseng, I. Csanueva, S. Ultes, O. Ramadan, and M. Gašić, "MultiWOZ – A large-scale multi-domain wizard-of-oz dataset for task-oriented dialogue modelling," in *Proceedings of EMNLP 2018*, pp. 5016–5026, 2018. [arXiv:1810.00278](https://arxiv.org/abs/1810.00278)
- [14] Groq, Inc., "Groq API documentation," 2024. [Online]. Available: <https://console.groq.com/docs>
- [15] D. Dubey, "Token-Efficient Conversational History Management using Sparse Semantic Patch Memory," Zenodo, Mar. 2026. [Online]. Available: <https://doi.org/10.5281/zenodo.19033643>